Module 16 - CSS

4. CSS Selectors & Styling

Que: 1

What is a CSS selector? Provide examples of element, class, and ID selectors.

Ans:

- **Meaning:**

  CSS selector is used to select HTML elements and apply styles to them.

- **Types with Examples:**

1. **Element Selector (select by tag name)**

   ```
   p {
     color: red;
   }
   ```

   - All <p> tags will become red.

2. **Class Selector (use .)**

   ```
   .box {
     background-color: yellow;
   }
   ```

   - All elements with class="box" will be styled.

3. **ID Selector (use #)**

   ```
   #header {
     font-size: 20px;
   }
   ```

   - Only element with id="header" will be styled.

Que: 2

Explain the concept of CSS specificity. How do conflicts between multiple styles get resolved?

Ans:

- **Meaning:**

  Specificity decides which CSS style will apply when multiple styles target the same element.

- **Priority Order (High → Low):**
  - Inline CSS
  - ID Selector (#id)
  - Class Selector (.class)
  - Element Selector (p, div)
  - Example:
    - p { color: blue; }
    - .text { color: green; }
    - #para { color: red; }
  - If all applied → Red will win (ID highest priority)
- **Important Points:**
  - More specific selector overrides less specific.
  - If same specificity → last one applied wins

Que: 3

What is the difference between internal, external, and inline CSS? Discuss the advantages and disadvantages of each approach.

Ans:

❖ **Difference between Internal, External, and Inline CSS**

1. **Inline CSS**

- **Definition:**

- CSS is applied directly inside HTML elements using the style attribute.

- **Example:**

  Hello

- **Advantages:**

- Quick and easy to use
- Useful for small or temporary changes

- **Disadvantages:**

- Not reusable
- Makes HTML code messy
- Hard to maintain

2. **Internal CSS**

- **Definition:**

- CSS is written inside <style> tag in the same HTML file.

- **Example:**

  <style>

  p { color: blue; }

  </style>

- **Advantages:**

- Good for single page websites
- No need for separate file

- **Disadvantages:**

- Cannot reuse in multiple pages
- Increases file size

3. **External CSS**

- **Definition:**

- CSS is written in a separate .css file and linked to HTML.

- **Example:**

  <link rel="stylesheet" href="style.css">

- **Advantages:**

- Reusable across multiple pages
- Clean and organized code

- Easy to maintain (Best practice)

- **Disadvantages:**

- Needs separate file
- Slight delay in loading

➢ **Key Difference (Short Table):**

Type	**Location**	**Use Case**
Inline	Inside HTML tag	Small/quick changes
Internal	Same HTML file	Single page
External	Separate CSS file	Large/multi-page

➢ **Final Conclusion:**

- External CSS is the best approach because it is reusable, maintainable, and keeps code clean.

5. CSS Box Model

Que: 1

Explain the CSS box model and its components (content, padding, border, margin). How does each affect the size of an element?

Ans:

❖ **CSS Box Model**

- **Definition:**

  The CSS Box Model is used to define the layout and space of an element. Every element is treated as a rectangular box.

➢ **Components of Box Model**

1. **Content**

- This is the actual content (text, image, etc.)
- Width and height apply to this area

- **Example:**
  width: 200px;
  height: 100px;

2. **Padding**

- Space inside the element, between content and border
- Increases the size of the element

- **Example:**
  padding: 10px;

3. **Border**

- Line surrounding the padding and content
- Also adds to the total size

- **Example:**
  border: 2px solid black;

4. **Margin**

- Space outside the element, between elements
- Does not affect inner size but increases space around element

- **Example:**
  margin: 20px;

➢ **How it affects size**

- **Total element size** = Content + Padding + Border + Margin
- **Example:**

  width: 200px;

  padding: 10px;

  border: 2px;

  margin: 20px;

  - Content = 200px
  - Padding = 20px (10px left + 10px right)

- Border = 4px (2px left + 2px right)
- Margin = 40px (20px left + 20px right)
- **Total width** = 200 + 20 + 4 + 40 = 264px

➢ **Important Point:**

- box-sizing: border-box; makes size easier because padding and border are included inside width.

➢ **Final Conclusion:**

- The box model controls spacing and layout. Padding and border increase element size, while margin adds space outside.

Que: 2

What is the difference between border-box and content-box box-sizing in CSS? Which is the default?

Ans:

❖ **Difference between border-box and content-box in CSS**

1. **Content-Box (Default)**

- **Meaning:**

- Width and height apply only to content
- Padding and border are added outside

- **Example:**
  width: 200px;
  padding: 10px;
  border: 5px solid;
  box-sizing: content-box;
- **Total width** = 200 (content) + 20 (padding) + 10 (border) = 230px

2. **Border-Box**

- **Meaning:**

- Width and height include content + padding + border
- No extra size is added outside

- **Example:**
  width: 200px;
  padding: 10px;
  border: 5px solid;
  box-sizing: border-box;
- **Total width** = 200px only (Content size automatically adjusts)

➢ **Key Difference:**

Property	**Content-Box**	**Border-Box**
Size includes	Only content	Content + padding + border
Total size	Increases	Fixed (no increase)
Layout control	Harder	Easier

➢ **Default Value:**

- **Content**-box is the default

➢ **Final Conclusion:**

- **Border**-box is preferred because it makes layout easier and predictable.

6. CSS Flexbox

Que: 1

What is CSS Flexbox, and how is it useful for layout design? Explain the terms flex-container and flex-item.

Ans:

❖ **What is CSS Flexbox?**

- **Definition:**
- CSS Flexbox is a layout system used to arrange elements in a row or column easily and make designs flexible and responsive.

❖ **Why Flexbox is useful?**

- Easy to align items (center, left, right)
- Helps create responsive layouts
- Controls spacing and direction
- Reduces use of complex CSS (like float)

❖ **Important Terms**

1. **Flex Container**

- The parent element where Flexbox is applied
- **Use:** display: flex;
- **Example:**

```
.container {
  display: flex;
}
```

2. **Flex Item**

- The child elements inside flex container
- Automatically follow flex rules
- **Example:**

```
<div class="container">
  <div>Item 1</div>
  <div>Item 2</div>
</div>
```

- **Here:**
  - .container = Flex Container
  - Item 1, Item 2 = Flex Items

❖ **Simple Example**

```
.container {
  display: flex;
```

```
  justify-content: center;
  align-items: center;
}
```

- Items will be centered horizontally and vertically

- **Final Conclusion:**
  - Flexbox makes layout design simple, flexible, and responsive, and is widely used in modern web design.

Que: 2

Describe the properties justify-content, align-items, and flex-direction used in Flexbox.

Ans:

❖ **Flexbox Properties**

1. **justify-content**

- **Meaning:**
- Controls horizontal alignment of items (main axis).
- **Common values:**
- flex-start → items at start
- flex-end → items at end
- center → items in center
- space-between → equal space between items
- space-around → space around items
- **Example:**

```
.container {
  display: flex;
  justify-content: center;
}
```

2. **align-items**

- **Meaning:**
- Controls vertical alignment of items (cross axis).
- **Common values:**
- flex-start → top
- flex-end → bottom
- center → vertically center
- stretch → stretch items
- **Example:**

```
.container {
  display: flex;
  align-items: center;
}
```

3. **flex-direction**

- **Meaning:**
- Defines direction of items (row or column).
- **Common values:**
- row → left to right (default)
- row-reverse → right to left
- column → top to bottom
- column-reverse → bottom to top

- **Example:**

```
.container {
  display: flex;
  flex-direction: row;
}
```

➢ **Important Point:**

- justify-content works on main axis
- align-items works on cross axis

➢ **Final Conclusion:**

- These properties help control alignment, spacing, and direction, making layout design easy and flexible.

7. CSS Grid

Que: 1

Explain CSS Grid and how it differs from Flexbox. When would you use Grid over Flexbox?

Ans:

❖ **What is CSS Grid?**

- **Definition:**

- CSS Grid is a layout system used to create 2D layouts (rows and columns) on a web page.
- It allows you to design complex layouts easily

- **Example:**

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

- Creates 3 equal columns

❖ **Difference between Grid and Flexbox**

Feature	Flexbox	Grid
Dimension	1D (row OR column)	2D (row AND column)
Layout control	Simple layouts	Complex layouts
Direction	One direction at a time	Both directions together
Use case	Alignment & spacing	Full page layout

❖ **When to use Grid vs Flexbox?**

➢ **Use Flexbox when:**

- You need alignment (center, space-between)
- Layout is simple (one row or one column)
- **Example:** Navbar, buttons, small sections

➢ **Use Grid when:**

- You need rows + columns together
- Layout is complex
- **Example:** Full website layout, dashboard, gallery

❖ **Final Conclusion:**

- Use Flexbox for simple layouts (1D)
- Use Grid for complex layouts (2D)

Que: 2

Describe the grid-template-columns, grid-template-rows, and grid-gap properties. Provide examples of how to use them.

Ans:

❖ **CSS Grid Properties**

1. **grid-template-columns**
   - **Meaning:**
   - Defines number and width of columns in a grid.
   - **Example:**

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 100px;
}
```

   - Creates 3 columns with fixed widths
   - **Another example:**

```
grid-template-columns: 1fr 1fr 1fr;
```

   - Creates 3 equal columns

2. **grid-template-rows**
   - **Meaning:**
   - Defines height of rows in a grid.
   - **Example:**

```
.container {
  display: grid;
  grid-template-rows: 100px 200px;
}
```

   - Creates 2 rows with given heights

3. **grid-gap (gap)**
   - **Meaning:**
   - Defines space between rows and columns
   - **Example:**

```
.container {
  display: grid;
  grid-gap: 10px;
}
```

- Adds 10px space between all items
- **You can also write:**
  gap: 10px 20px;
- Row gap = 10px, Column gap = 20px

➢ **Combined Example**

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: 100px 100px;
  gap: 10px;
}
```

- **Output:**
  - 2 columns
  - 2 rows
  - 10px space between items

➢ **Final Conclusion:**

- These properties help control structure (rows & columns) and spacing, making grid layouts easy to design.

8. Responsive Web Design with Media Queries

Que: 1

What are media queries in CSS, and why are they important for responsive design?

Ans:

❖ **What are Media Queries in CSS?**

- **Definition:**
  - Media queries are used to apply CSS styles based on screen size or device type.
  - They help websites adjust layout for mobile, tablet, and desktop
- **Example:**

```
@media (max-width: 600px) {
  body {
    background-color: lightblue;
  }
}
```

  - When screen width is 600px or less, style will change

➢ **Why are Media Queries Important?**

- Make website responsive 📱💻
- Improve user experience on different devices
- Adjust layout, font size, images
- Support mobile-friendly design

➢ **Common Breakpoints:**

- Mobile → up to 600px
- Tablet → 601px to 1024px
- Desktop → 1025px and above

➢ **Final Conclusion:**

- Media queries are important because they help create responsive websites that work on all screen sizes.

Que: 2

Write a basic media query that adjusts the font size of a webpage for screens smaller than 600px.

Ans:

- **Basic Media Query Example**
  @media (max-width: 600px) { body {   font-size: 14px; }}
- **Explanation:**
  - @media (max-width: 600px) → applies style when screen is 600px or smaller
  - font-size: 14px; → reduces text size for small screens (mobile)
- **Final Line:**

- This media query helps make text readable on smaller devices like mobile phones.

9. Typography and Web Fonts

Que: 1

Explain the difference between web-safe fonts and custom web fonts. Why might you use a web-safe font over a custom font?

Ans:

❖ **Difference between Web-safe Fonts and Custom Web Fonts**

1. **Web-safe Fonts**
   - **Definition:**
   - Fonts that are already installed on most devices and browsers.
   - Examples: Arial, Times New Roman, Verdana
   - **Advantages:**
   - Load very fast
   - Supported on all devices
   - No need to download
   - **Disadvantages:**
   - Limited design options
   - Less unique look
2. **Custom Web Fonts**
   - **Definition:**
   - Fonts that are not installed by default and are loaded using CSS (e.g., from Google Fonts).
   - **Example:**

     @import url('https://fonts.googleapis.com/css2?family=Roboto&display=swap');

   - **Advantages:**
   - More creative and unique design
   - Better typography options
   - **Disadvantages:**
   - Requires internet to load
   - Can slow down website
   - Why use Web-safe Fonts over Custom Fonts?
   - Faster loading speed
   - Better compatibility
   - Works even without internet

➢ **Final Conclusion:**

- Use web-safe fonts for speed and reliability, and custom fonts for better design and style.

Que: 2

What is the font-family property in CSS? How do you apply a custom Google Font to a webpage?

Ans:

❖ **What is font-family in CSS?**

- **Definition:**

- font-family is used to set the font style of text in a webpage.
- It also allows fallback fonts (backup fonts if first one is not available)

- **Example:**

```
p {
  font-family: Arial, sans-serif;
}
```

- If Arial is not available, browser will use sans-serif

❖ **How to Apply a Custom Google Font?**

- **Step 1: Add Google Font link in HTML**

```
<link href=https://fonts.googleapis.com/css2?family=Roboto&display=swap
rel="stylesheet">
```

- **Step 2: Use it in CSS**

```
body {
  font-family: 'Roboto', sans-serif;
}
```

➢ **Important Points:**

- Always add fallback font (like sans-serif)
- Custom fonts need internet to load

➢ **Final Conclusion:**

- font-family controls text style, and Google Fonts help make websites look more attractive.